Recursion

Recursive functions

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function. Syntax:

```
def foo(some_param):
    if some_param:
        foo(some_param)
```

or

```
def foo(some_param):
    if base_case_condition:
        return some_value
    else:
        foo(some_param)
```

Recursion has two main phases:

1. Base case: The condition when the recursion ends.
2. Recursive case: The condition when the function calls itself.

It is used to solve problems that can be broken down into smaller, repetitive problems.

Every recursive function must have a base case to stop the recursion that would otherwise go on indefinitely. If the base case is not reached or not defined, then the stack overflow problem may arise.

```
RecursionError: maximum recursion depth exceeded in comparison
```

In [ ]:
```
def print_recursive(n):
    if n <= 0:
        return
    print(n, end=' ')
    print_recursive(n-1)

print_recursive(3)
```

```
3 2 1
```

The factorial of a non-negative integer n:

$$n! = n * (n-1) * (n-2) * ... * 1 = n * (n-1)!$$

In [ ]:
```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0 or n == 1:
        print("1")
        return 1
    print(f"{n} * ", end=" ")
    return n * factorial(n-1)
```

```
factorial(5)
```

```
5 *  4 *  3 *  2 *  1
```

Out[ ]: 120

The sum of the first n natural numbers:

```
sum(n) = n + (n-1) + (n-2) + ... + 1 = n + sum(n-1)
```

In [ ]:
```
def sum_recursive(n):
    if n == 1:
        return 1
    return n + sum_recursive(n-1)

sum_recursive(15)
```

Out[ ]: 120

The Fibonacci sequence:
Each number is the sum of the two preceding ones, starting from 0 and 1.

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
= 0, 1, 0+1, 0+1+1, 0+1+1+2, 0+1+1+2+3, ...
= 0, 1, f(0)+f(1), f(1)+f(2), f(2)+f(3), f(3)+f(4), ...
```

In [ ]:
```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n-1) + fibonacci(n-2)

for i in range(10):
    print(fibonacci(i), end=' ')
```

```
0 1 1 2 3 5 8 13 21 34
```

Head recursion and tail recursion

Head recursion:
The recursive call is the first statement in the function. It is similar to the iteration process.

In [ ]:
```
def head_recursive(n):
    if n > 0:
        head_recursive(n-1)
        print(n, end=' ')

head_recursive(5)
```

```
1 2 3 4 5
```

Tail recursion:
The recursive call is the last statement in the function.

In [ ]:
```python
def tail_recursive(n):
    if n > 0:
        print(n, end=' ')
        tail_recursive(n-1)

tail_recursive(5)
```

```
5 4 3 2 1
```

Finite recursion and infinite recursion

Finite recursion

It is when the recursive function reaches the base case after a finite number of recursive calls.

Infinite recursion

It is when the base case is never reached.

Infinite recursion can be avoided by ensuring that the base case is reachable.

In [ ]:
```python
def infinite_recursive(n):
    print(n)
    infinite_recursive(n+1)

# infinite_recursive(0)
# RecursionError: maximum recursion depth exceeded in comparison
```

Direct and indirect recursion

Direct recursion:

A function calls itself directly.

```python
def foo():
    foo()
```

Indirect recursion:

A function calls another function, which in turn calls the first function directly or indirectly.

```python
def foo():
    bar()

def bar():
    foo()
```

In [ ]:
```python
def indirect_foo_a(n):
    if n > 0:
        print(f"a{n}", end=' ')
        indirect_foo_b(n//2)

def indirect_foo_b(n):
    if n > 1:
        print(f"b{n}", end=' ')
        indirect_foo_a(n-1)
```

```
indirect_foo_a(20)
```

```
a20 b10 a9 b4 a3
```

Nested recursion

In nested recursion, a recursive function is passed as a parameter to another recursive function.

In [ ]:
```
def nested_rec(n):
    if n > 100:
        return n - 10
    return nested_rec(nested_rec(n+11))

nested_rec(95)
```

Out[ ]:
```
91
```

Here, the function `nested_rec` is passed as a parameter to itself.

```
nested_rec(95) = nested_rec(nested_rec(106)) = nested_rec(96)
nested_rec(96) = nested_rec(nested_rec(107)) = nested_rec(97)
nested_rec(97) = nested_rec(nested_rec(108)) = nested_rec(98)
nested_rec(98) = nested_rec(nested_rec(109)) = nested_rec(99)
nested_rec(99) = nested_rec(nested_rec(110)) = nested_rec(100)
nested_rec(100) = nested_rec(nested_rec(111)) = nested_rec(101)
nested_rec(101) = 91
```